Cascading Style Sheets (CSS) — Dressing Up the Web

IWT — Week 6

> **The story in one breath:** HTML is the architect's blueprint — it says *there is a wall here, a door there*. CSS is the interior decorator — it decides the paint, the spacing, the furniture. And when two decorators give conflicting orders about the same wall, the **cascade** is the foreman who decides whose order wins.

Index

- 1. What CSS Does and Why It Matters — *why your HTML does not have to wear the same grey outfit forever.*
- 2. Anatomy of a CSS Rule — *selector, braces, property, value: the grammar of fashion.*
- 3. Where Styles Live — *inline, internal, external: a studio flat, a house, or a whole city.*
- 4. The Property Catalogue — *the wardrobe: fonts, colours, borders, spacing and friends.*
- 5. Colours in CSS — *five ways to say "red" without saying "red".*
- 6. Units in CSS — *rulers that stretch, rulers that don't.*
- 7. The HTML Skeleton and Semantic Tags — *every page has bones; some bones have names.*
- 8. The Document Tree and Scoping of Selectors — *a family tree where tags argue about who is whose parent.*
- 9. Selectors Decide Which Elements a Rule Targets — *the art of pointing at exactly the right element.*
- 10. When Rules Conflict, Cascade Decides — *inheritance, specificity, location: the court of appeals.*
- 11. Practice Questions — *five puzzles that refuse to be answered by memorising.*

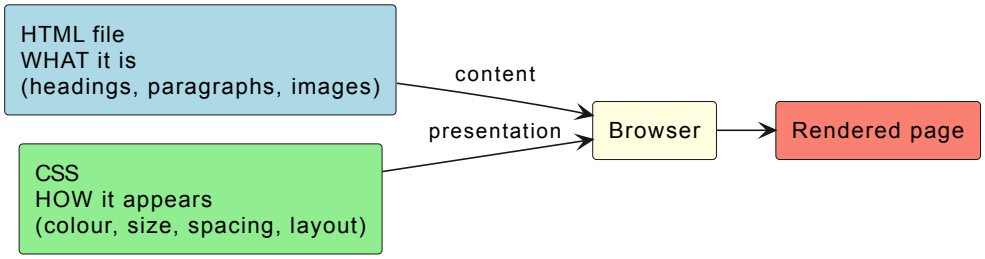
---

1. What CSS Does and Why It Matters

CSS stands for **Cascading Style Sheets**, a **W3C standard** for describing the appearance of HTML elements.

HTML gives you the *container* — the basic template that organises your web content on a page. CSS sits **on top of HTML** and adds everything about presentation: how the content looks, how the visuals come together, and how the experience feels. All of that is dictated by CSS.

- It **separates content from presentation** — HTML says *what something is*, CSS says *how it appears*.
- It can be added **directly to any HTML element** (via the `style` attribute), **within the `<head>` element**, or, most commonly, in a **separate text file that contains only CSS**.
- **One stylesheet can style an entire site**, which is really what makes a codebase maintainable.
- It is also the mechanism behind **responsive design** — the same markup can look different on every device.

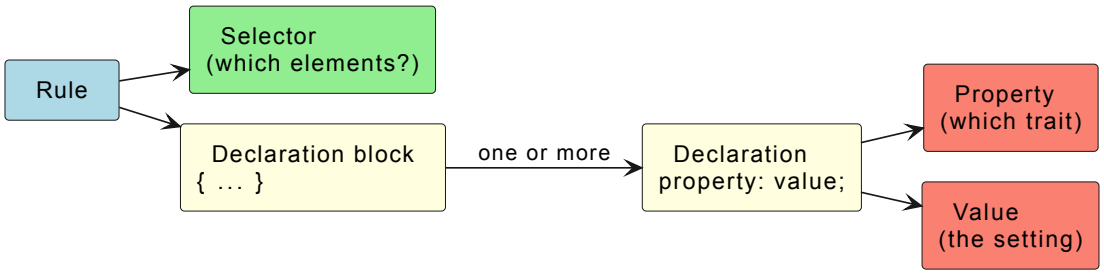


Aspect	HTML	CSS
Role	Structure and meaning	Appearance and layout
Question it answers	"What is this element?"	"How should it look?"
Change the look of 100 pages	Edit 100 files	Edit 1 stylesheet

2. Anatomy of a CSS Rule

A CSS document consists of one or more **style rules**. A rule consists of a **selector** that identifies the HTML element or elements that will be affected, followed by a series of **property: value pairs**.

- The **selector** selects the HTML element(s) on which the styling will apply.
- The **declaration block** is the scope, enclosed in two braces `{ }`.
- Inside the block, each **declaration** is a `property: value` pair, and multiple declarations are separated by a delimiter — the **semicolon**.



General shape of a rule:

```
selector {
  property: value;
  property2: value2;
}
```

Example 1 — an emphasis element. Whenever the HTML contains an `em` element, its colour is set to red.

```
em { color: red; }
```

Here `em` is the selector, `color` is the property and `red` is the value.

Example 2 — a paragraph. Whenever there is a `p` element, the following margin, font weight and font family apply.

```
p {
  margin: 5px 0 10px 0;
  font-weight: bold;
  font-family: Arial, Helvetica, sans-serif;
}
```

Reading the `margin` shorthand step by step (order is **top, right, bottom, left**):

Value	Side	Meaning
`5px`	top	5 pixels of space above
`0`	right	none
`10px`	bottom	10 pixels of space below
`0`	left	none

`font-family` lists fonts in order of preference: try Arial, then Helvetica, and if neither exists use any generic `sans-serif`.

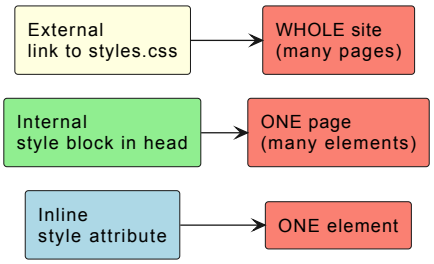
3. Where Styles Live

The same rule can be placed in three locations. The location decides the **scope** (how much of your content it reaches), and what is convenient in one use case may not be in another.

```
h1 { color: navy; }
```

Part	Role
`h1`	**selector** — what to style
`color`	**property** — which trait
`navy`	**value** — the setting

Location	Code	Scope
Inline	`<h2 style="color:red;">`	one element, via the `style` attribute
Internal	`<style> h2{color:red;} </style>` (in `<head>`)	the whole page, in a `<style>` block
External	`<link rel="stylesheet" href="styles.css">`	the whole site — one file, usually the best choice



- **Inline:** the rule lives inside the HTML element itself. The property and value pair is written inside double quotes, with a colon between them. The scope is limited to *only that element*.
- **Internal:** a block structure — opening tag `<style>`, closing tag `</style>` — placed in the `<head>`. Several elements can share the styling, but it is still internal to that one HTML file.
- **External:** a separate file with a `.css` extension, connected using `link rel="stylesheet"`, where `href` gives the location of the sheet (here `styles.css`). All rule sets live in that file, outside the HTML. Any number of pages across the whole website can link to the same sheet, so it gives the **broadest scope** and is the most common choice.

> **Note for Review** Pro-Tip: Exams love the question "which location gives the widest scope?" Remember the ladder **element → page → site = inline → internal → external**. The wider the scope, the easier maintenance becomes.

4. The Property Catalogue

CSS offers a large set of properties, grouped by what they control. Properties marked with `*` are shorthand properties that bundle several related properties together (for example, `border` sets width, style and colour at once).

Property Type	Properties
Fonts	`font`, `font-family`, `font-size`, `font-style`, `font-weight`, `@font-face`
Text	`letter-spacing`, `line-height`, `text-align`, `text-decoration*`, `text-indent`
Color and background	`background`, `background-color`, `background-image`, `background-position`, `background-repeat`, `box-shadow`, `color`, `opacity`
Borders	`border*`, `border-color`, `border-width`, `border-style`, `border-top`, `border-left`, … `*`, `border-image*`, `border-radius`
Spacing	`padding`, `padding-bottom`, `padding-left`, …, `margin`, `margin-bottom`, `margin-left`, …
Sizing	`height`, `max-height`, `max-width`, `min-height`, `min-width`, `width`
Layout	`bottom`, `left`, `right`, `top`, `clear`, `display`, `float`, `overflow`, `position`, `visibility`, `z-index`
Lists	`list-style*`, `list-style-image`, `list-style-type`
Effects	`animation*`, `filter`, `perspective`, `transform*`, `transition*`

What each family lets you do:

- **Fonts and text:** choose the typeface, size and weight; control alignment, line height, indentation and spacing between letters.
- **Color and background:** set background colour, image, position, and whether the image repeats to cover the whole area; add a shadow; make an element transparent or opaque.
- **Borders:** control width, colour and style (continuous or dotted line); style only the top, left or right border; use a border image or rounded corners.
- **Spacing:** `margin` is the space *outside* an element; `padding` is the space *inside* it, between content and border.
- **Sizing:** fix or limit the height and width, with minimum and maximum bounds.
- **Layout:** position an element (bottom, top, left, right), choose its `display` behaviour, make it `float`, control `visibility` and stacking order with `z-index`.
- **Lists and effects:** bullet style and image; animations, filters, transforms and transitions.

---

5. Colours in CSS

Different notations suit different purposes, so explore which fits your need.

Method	Description	Example
Name	One of 17 standard colour names. CSS3 has 140 standard names.	`color: red;` and `color: hotpink; /* CSS3 only */`
RGB	Three numbers between 0 and 255 for the red, green and blue values.	`color: rgb(255,0,0);` and `color: rgb(255,105,180);`
Hexadecimal	A six-digit hexadecimal number for the red, green and blue values; each of the three is between 0 and FF (255 in decimal). Preceded by a hash `#`.	`color: #FF0000;` and `color: #FF69B4;`
RGBa	A partially transparent colour. The "a" stands for *alpha*, a value between 0.0 (fully transparent) and 1.0 (fully opaque).	`color: rgba(255,0,0,0.5);`
HSL	Specify a colour using Hue, Saturation and Light values. Available only in CSS3; HSLA is available as well.	`color: hsl(0,100%,100%);` and `color: hsla(330,59%,100%,0.5);`

Why 0 to 255? Each channel is stored in **8 bits**, so it has $2^8 = 256$ possible levels, numbered 0 to 255. Mixing red, green and blue in different proportions gives every shade.

$$2^8 = 256 \quad \Rightarrow \quad \text{levels } 0, 1, \ldots, 255$$

Converting RGB to hexadecimal. Each channel becomes two hex digits: divide by 16 for the first digit, the remainder is the second.

$$255_{10} = 15 \times 16 + 15 = \mathrm{FF}_{16}$$

$$105_{10} = 6 \times 16 + 9 = 69_{16}$$

$$180_{10} = 11 \times 16 + 4 = \mathrm{B4}_{16}$$

Therefore `rgb(255,0,0)` = `#FF0000` (pure red) and `rgb(255,105,180)` = `#FF69B4` (hotpink). Channels with value 0 become `00`.

> **Note for Review** Pro-Tip: In HSL the hue is an angle on the colour wheel (0 is red, 330 is a pink-magenta), saturation is the colour intensity and lightness runs from black (0%) to white (100%). The slide example `hsl(0,100%,100%)` therefore has maximum lightness and actually renders **white**; a pure red would be `hsl(0,100%,50%)`. Treat the slide values as syntax illustrations.

6. Units in CSS

Whenever you specify spacing, padding or size, you must give a unit. The smallest unit of measure is used for computation — whether you are measuring a distance, the basic unit used to render graphics, or resizing a box relative to another size. Units are either **absolute** (fixed) or **relative** (depend on something else).

Unit	Description	Type
`px`	Pixel. In CSS2 this is a relative measure, while in CSS3 it is absolute (1/96 of an inch).	Relative (CSS2), Absolute (CSS3)
`em`	Equal to the computed value of the `font-size` property of the element on which it is used. When used for font sizes, `em` is in relation to the font size of the parent.	Relative
`%`	Always relative to another value. The precise meaning of `%` varies depending upon the property in which it is used.	Relative
`ex`	A rarely used measure relative to the x-height of an element's font.	Relative
`ch`	A rarely used measure relative to the width of the zero ("0") character of an element's font.	Relative (CSS3 only)
`rem`	*Root em* — the font size of the root element. Unlike `em`, which may differ for each element, `rem` is constant throughout the document.	Relative (CSS3 only)
`vw`, `vh`	Viewport width and viewport height, as percentage values (between 0 and 100) of the viewport (browser window). An item changes size when the viewport is resized.	Relative (CSS3 only)
`in`	Inches	Absolute
`cm`	Centimeters	Absolute
`mm`	Millimeters	Absolute
`pt`	Points (equal to 1/72 of an inch)	Absolute
`pc`	Pica (equal to 1/6 of an inch)	Absolute

Absolute units are all linked through the inch. With 1 in = 96 px:

$$1\,\text{pt} = \frac{1}{72}\,\text{in} = \frac{96}{72}\,\text{px} \approx 1.33\,\text{px}$$

$$1\,\text{pc} = \frac{1}{6}\,\text{in} = 16\,\text{px} = 12\,\text{pt}$$

`em` vs `rem` by example. Suppose the root font size is $16\,\text{px}$.

$$2\,\text{rem} = 2 \times 16 = 32\,\text{px} \quad \text{(always, anywhere in the document)}$$

If an element's parent has `font-size: 20px`, then `font-size: 1.5em` on the child gives:

$$1.5\,\text{em} = 1.5 \times 20 = 30\,\text{px}$$

> **Note for Review** Pro-Tip: `em` compounds when elements are nested (each level multiplies the parent's size), while `rem` always refers back to the root. That is why `rem` is the predictable choice for consistent spacing, and `%`, `vw`, `vh` are the tools for fluid, resizable layouts.

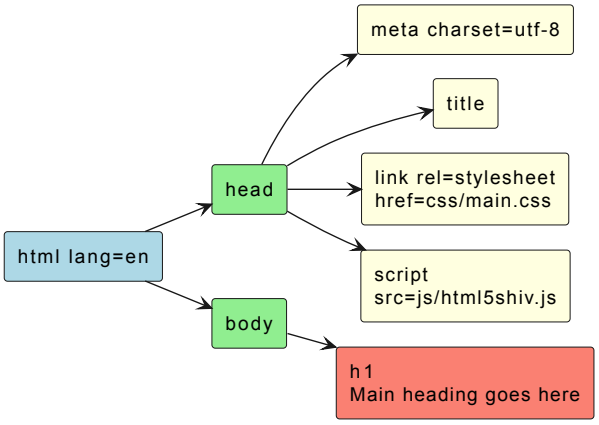
7. The HTML Skeleton and Semantic Tags

7.1 Every HTML Document Follows the Same Skeleton

Before styling anything, recall the structure CSS sits on. Every HTML document has the same skeleton:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Share Your Travels -- New York - Central Park</title>
  <link rel="stylesheet" href="css/main.css" />
  <script src="js/html5shiv.js"></script>
</head>
<body>
  <h1>Main heading goes here</h1>
  ...
</body>
</html>
```

No.	Part	Purpose
1	`<!DOCTYPE html>`	The outermost declaration; tells the browser this is an HTML document. It begins the first block in the hierarchy.
2	`<html lang="en">` ... `</html>`	The root element; everything else is nested inside it, which gives the document its scope.
3	`<head>`	Information *about* the page (not displayed as content).
4	`<body>`	The visible content of the page.
5	`<meta charset="utf-8" />`	Declares the character encoding.
6	`<link rel="stylesheet" href="css/main.css" />`	Connects the external CSS file.
7	`<script src="js/html5shiv.js"></script>`	Loads an external JavaScript file.



7.2 Semantic Tags Describe Meaning, Not Just Boxes

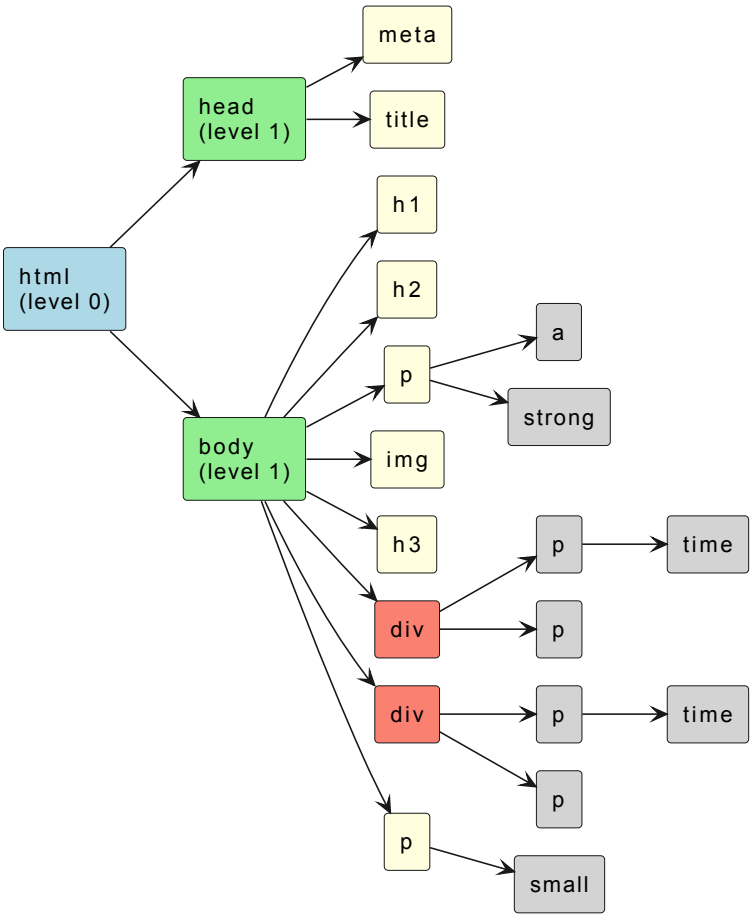
You *could* build a page out of anonymous boxes (`div`) and name them with classes. A better way is to use tags whose names state what the region **actually means**. The visual page is the same, but one naming scheme is meaningful to browsers, screen readers and search engines.

How you might name it	What it actually means
`<div class="header">`	`<header>`
`<div class="nav">`	`<nav>`
`<div class="main">`	`<main>` / `<article>`
`<div class="footer">`	`<footer>`

8. The Document Tree and Scoping of Selectors

An HTML file always maintains a **hierarchy**. Between `<head>` and `</head>`, and between `<body>` and `</body>`, you place nested elements: inside `body` you may have `h1`, `p`, `img`, `h3`, `div`, and so on. Because elements nest like this, the document forms a **tree** (a directed acyclic graph: there are **no cycles**). The **root** is `html`, so the entire scope is driven by it.

Tags open and close like brackets, and whatever sits between an opening and a closing tag lies in that element's scope. This is the same idea as the page layout (header, navigation, main article, footer) nested inside the outermost `html` block.



Family vocabulary (applies to any elements arranged in a tree):

Relation	Definition	Example from the tree
Parent / child	A direct link one level apart	`head` is the parent of `meta` and `title`; `title` is a child of `head`
Siblings	Elements that **share the same parent** (same level, same scope)	`head` and `body` (both level 1, parent `html`); `meta` and `title`
Ancestor / descendant	Anywhere along a chain of parent-child links	`time` is a descendant of both `p` and `div`
Root / levels	The top of the tree is level 0; each step down adds one level	`html` is level 0; `head`, `body` are level 1

Some nestings are fixed by how HTML works: an `a` element sits under a `p` scope, and `small` always sits under `p`. The `div` elements, `h1`, `h2`, `p`, `img` and `h3` all share the same parent `body`, yet each `div` has its **own scope**, rooted at that point.

Why this matters for CSS: the scope of a rule set is read straight off this tree. A selector's job is to pick an element and apply the rule set to it, and the tree decides which elements are reached when we select by position (parent, child, descendant, sibling).

9. Selectors Decide Which Elements a Rule Targets

Selectors provide a **handle** that tells the browser which HTML elements will be affected by the property values. The property could be anything (size, font, colour), and the value must come from the permitted domain of that property. For `color`, only certain values are admissible, but there are many notations (`red`, or `rgb(255,0,0)`).

Different selector types exist so you can control the scope: pick **one** element; pick an element **and everything rooted beneath it**; or apply rules only when a specific type is found (only `div`, only `p`, only images). Access can be by **type, class, name, attribute, or context (hierarchy)**.

9.1 Element, Class and ID Selectors

- **Element selector** — `p { }` matches **every** `<p>` on the page. This is the most basic selector.
- **Class selector** — allows you to **simultaneously target different HTML elements regardless of their position in the document tree**.
  - `.highlight { }` matches any element carrying `class="highlight"`.
  - A class is a tag you attach to many elements so they can be handled together.
- **ID selector** — allows you to target **a specific element** by its `id` attribute regardless of its type or position.
  - `#nav { }` matches the one element with `id="nav"` (**ids should be unique**).

Selector	Syntax	Matches	Uniqueness
Element	`p { }`	all `<p>` elements	n/a
Class	`.highlight { }`	every element with that class	reusable on many elements
ID	`#nav { }`	the single element with that id	**one per page**

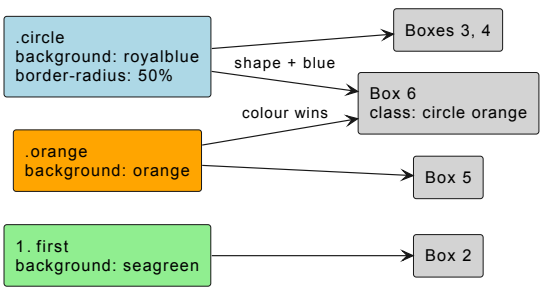
9.2 Worked Example — ID and Class Together

Consider seven boxes. Without any styling (just plain `div`s) they all look the same grey. Now organise the HTML with an `id` and some classes. (These attributes belong to the **HTML**; the CSS below targets them.)

```
<div></div>
<div id="first"></div>
<div class="circle"></div>
<div class="circle"></div>
<div class="orange"></div>
<div class="circle orange"></div>
<div></div>
```

```
#first {
  background-color: seagreen;
}
.circle {
  background-color: royalblue;
  border-radius: 50%;
}
.orange {
  background-color: orange;
}
```

Box	HTML	Result
1	`<div></div>`	grey square (no rule matches)
2	`id="first"`	**green** square (`#first`)
3	`class="circle"`	**blue circle** (`.circle`)
4	`class="circle"`	**blue circle** (`.circle`)
5	`class="orange"`	**orange** square (`.orange`)
6	`class="circle orange"`	**orange circle**
7	`<div></div>`	grey square



An element can belong to **multiple classes** — box 6 is a circle *and* orange, so both rule sets apply. But both set `background-color`, and an element cannot be blue and orange at once. Some mechanism must decide which rule persists and which is overridden; here `.circle` and `.orange` are equally specific, and `.orange` is written **later**, so the circle takes the orange colour while keeping the `border-radius`. The general machinery is the cascade (section 10).

9.3 Attribute Selectors

An **attribute selector** selects HTML elements either by the **presence** of an attribute or by the **value** of an attribute.

Selector	Matches	Example	Meaning of the example
`[]`	A specific attribute	`[title]`	any element with a `title` attribute
`[=]`	A specific attribute with a specific value	`a[title="posts from this country"]`	any `<a>` whose title is **exactly** "posts from this country"
`[~=]`	A specific attribute whose value matches at least one of the words in a space-delimited list of words	`[title~="Countries"]`	any `title` attribute that contains the **word** "Countries"
`[^=]`	A specific attribute whose value **begins with** a specified value	`a[href^="mailto"]`	any `<a>` whose `href` begins with "mailto"
`[*=]`	A specific attribute whose value **contains a substring**	`img[src*="flag"]`	any `<img>` whose `src` contains somewhere within it the text "flag"
`[$=]`	A specific attribute whose value **ends with** a specified value	`a[href$=".pdf"]`	any `<a>` whose `href` ends with the text ".pdf"

When the pattern matches, the rule set applies — and that rule set can be anything: a font, colour, border or size.

Example A — form inputs. Style only `<input>` elements whose `type` is exactly `text`:

```
input[type="text"] {
  height: 40px;
  border-style: none;
  border-bottom: 2px solid blue;
  background-color: beige;
}
```

HTML element	Styled?
`<input type="text" />` (two of them)	**Yes**
`<input type="radio">`	No
`<input type="checkbox">`	No
`<input type="password">`	No

Example B — PDF links. Style only `<a>` elements whose `href` ends with `.pdf`, adding an icon (an SVG file stored elsewhere and embedded as a background image):

```
a[href$=".pdf"] {
  background: url(pdf_icon.svg) no-repeat left center;
  padding-left: 19px;
}
```

HTML	Styled?
`<a href="abc.html">First link</a>`	No — plain link
`<a href="sample.pdf">One PDF</a>`	**Yes** — gets the PDF icon
`<a href="another.pdf">Two PDF</a>`	**Yes** — gets the PDF icon

The `padding-left: 19px` makes room so the text does not sit on top of the icon.

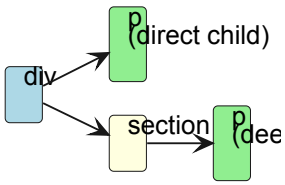
> **Note for Review** Pro-Tip: Do not mix up `~=`, `*=`, `^=`, `$=`. Memory hook: `~` = whole **word** in a list, `*` = **anywhere** inside, `^` = **starts** (caret points to the front), `$` = **ends** (as in regular expressions).

9.4 Contextual Selectors (Combinators)

A **contextual selector** (in CSS3 also called **combinators**) lets you select elements based on their **ancestors, descendants, or siblings** in the document tree. It was added later, but it gives a lot of flexibility.

Selector	Matches	Example	Meaning
Descendant	A specified element contained somewhere within another specified element	`div p`	a `<p>` contained somewhere within a `<div>`; it can be any descendant, not just a child
Child	A specified element that is a direct child of the specified element	`div>h2`	an `<h2>` that is a child of a `<div>`
Adjacent sibling	A specified element that is the next sibling (comes directly after)	`h3+p`	the **first** `<p>` after any `<h3>`
General sibling	All following elements that share the same parent as the specified element	`h3~p`	**all** `<p>` elements after an `<h3>` that share the same parent as the `<h3>`

In short: `div p { }` matches any `<p>` inside a `<div>`; `div > p { }` matches **only direct children**.



- `div p` selects **both** `p` elements: any element in the rooted sub-tree under the `div` is a descendant, however long the chain of parent-child links to reach it.
- `div > p` selects **only the first** `p`: it must be a direct child, one level below.

For siblings, consider the tree under `body`: an `h3` followed immediately by a `p` makes them **adjacent** siblings (`h3+p` hits that `p`). A `p` that comes later under the same parent, but with other elements in between, is not adjacent, yet it is still a **general** sibling of the `h3` (`h3~p` hits it). Likewise `h2` and `p` under the same parent are general siblings, not adjacent ones.

Selector	Picks	Needs same parent?	Needs to be directly next?
`div p`	any descendant `p`	no	no
`div>p`	direct child `p` only	yes (the div)	no
`h3+p`	the first `p` right after `h3`	yes	**yes**
`h3~p`	every later `p` sibling	yes	no

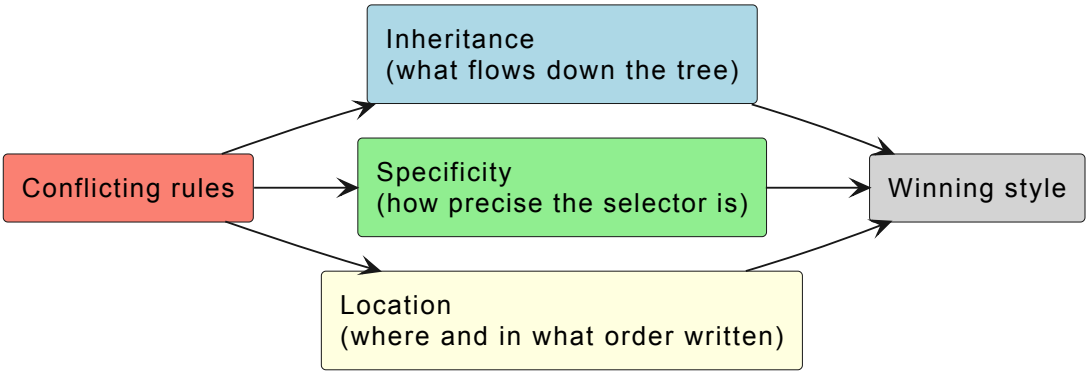
10. When Rules Conflict, Cascade Decides

Sooner or later two rules will tell the same element to look different. The **"Cascade"** in CSS refers to **how conflicting rules are handled**. This priority order is set by the W3C.

10.1 Three Sources of Style

Source	Description
Author-created style	CSS written by the web developer to style their site; it is what most people mean when they say "CSS".
User style sheets	Custom CSS a visitor applies in their own browser (client side) to override how sites look for them, e.g. for **accessibility or preference**.
Browser style sheets	The default styles a browser applies to HTML elements when no other CSS is given (also called **"user agent styles"**).

CSS uses three cascade principles to deal with conflicts: **inheritance**, **specificity**, and **location**.



10.2 Inheritance

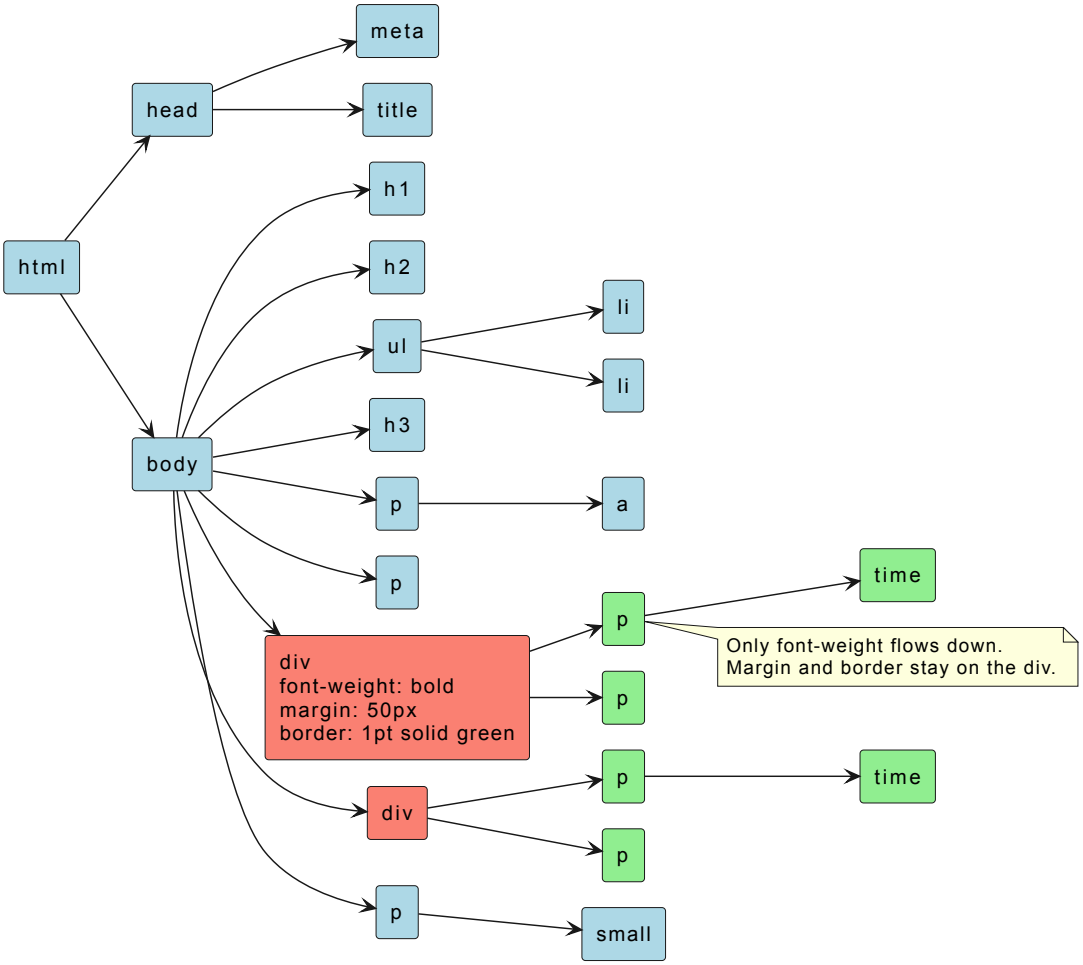
Some properties flow **down** from an element to its descendants; others do not.

- **Inheritable:** font, color, list and text properties.
- **Not inheritable:** layout, sizing, border, background and spacing properties.

Apply a rule to an ancestor and the inheritable properties follow automatically to every descendant. Consider this rule on `div`:

```
div {
  font-weight: bold;    /* inherited */
  margin: 50px;         /* not inherited */
  border: 1pt solid green;   /* not inherited */
}
```

The `p` elements inside the `div` **inherit the font weight** but **not the margin and not the border**.



Forcing inheritance with the keyword `inherit`. A non-inheritable property can still be copied from the parent if you ask explicitly:

```
div {
  font-weight: bold;
  margin: 50px;
  border: 1pt solid green;
}
p {
  border: inherit;
  margin: inherit;
}
```

```
<h3>Reviews</h3>
<div>
  <p>By Ricardo on <time>2016-05-23</time></p>
  <p>Easy on the HDR buddy.</p>
</div>
<hr/>

<div>
  <p>By Susan on <time>2016-11-18</time></p>
  <p>I love Central Park.</p>
</div>
<hr/>
```

Now each `p` copies the `div`'s `border` (1pt solid green) and `margin` (50px). In the rendered page every paragraph appears in its own green-bordered box, nested inside the `div`'s larger green box with extra spacing.

> **Note for Review** Pro-Tip: Exam favourite — "which of these properties is inherited?" Rule of thumb: anything about the **text itself** (font, colour, list, text) is inherited; anything about the **box around it** (layout, size, border, background, spacing) is not. A box belongs to one element; text style is naturally shared by its children.

10.3 Specificity

Inheritance says what *flows down*. Specificity says what *wins* when several rules directly target the same element. The nearest, most precise rule takes the style.

Worked example. Here is a portion of an HTML file (just a fragment, not the entire document) with its style sheet:

```
body {
  font-weight: bold;
  color: red;
}
div {
  font-weight: normal;
  color: magenta;
}
p {
  color: green;
}
.last {
  color: blue;
}
#verylast {
  color: orange;
  font-size: 16pt;
}
```

```
<body>
  This text is not within a p element.
  <p>Reviews</p>
  <div>
    <p>By Ricardo on <time>2016-05-23</time></p>
    <p>Easy on the HDR buddy.</p>
    This text is not within a p element.
  </div>
  <hr/>

  <div>
    <p>By Susan on <time>2016-11-18</time></p>
    <p>I love Central Park.</p>
  </div>
  <hr/>

  <div>
    <p class="last">By Dave on <time>2016-11-24</time></p>
    <p class="last" id="verylast">Thanks for posting.</p>
  </div>
  <hr/>
</body>
```

Since font and colour are inheritable, `body`'s bold red might be expected everywhere — but it is not, because more specific rules exist closer to each element:

Text	Colour	Weight	Why
"This text is not within a p element." (directly in `body`)	**red**	bold	only the `body` rule applies
"Reviews" (`p` under `body`)	**green**	bold	`p { color: green }` beats the inherited red; weight is still inherited from `body`
Ricardo's and Susan's `p` lines	**green**	normal	`p` rule sets the colour; `div` supplies `normal` weight
"This text is not within a p element." (inside the `div`)	**magenta**	normal	not a `p`, so the `div` rule applies
"By Dave on ..." (`class="last"`)	**blue**	normal	`.last` beats `p`
"Thanks for posting." (`class="last"` and `id="verylast"`)	**orange**, 16pt	normal	`#verylast` beats `.last`

The guiding idea: **whatever is local, whatever is nearest and most precise, wins.** A `p` inside a `div` inherits the `div`'s properties, but a rule written specifically for `p` overrides it. The class `last` makes text blue, but an element that also carries a very specific `id` is overridden by the id rule.

Specificity values. Each kind of selector carries a weight. A higher specificity value overrides a lower one.

Rank	Selector type	Example	Specificity value
1 (strongest)	Inline style attribute	`<div style="color: red;">`	1000
2	id + additional selectors	`div #firstExample { color: grey; }`	0101
3	id selector	`#firstExample { color: magenta; }`	0100
4	class and attribute selectors	`.example { color: blue; }` and `a[href$=".pdf"] { color: blue; }`	0010
5	descendant selector (elements only)	`div form { color: orange; }`	0002
6 (weakest)	element selector	`div { color: green; }`	0001

Each row overrides every row below it: inline overrides id + additional selectors, which overrides a lone id, which overrides class and attribute selectors, which override descendant selectors, which override a plain element selector.

Think of the four digits as four columns (a, b, c, d):

$$S = 1000\,a + 100\,b + 10\,c + d$$

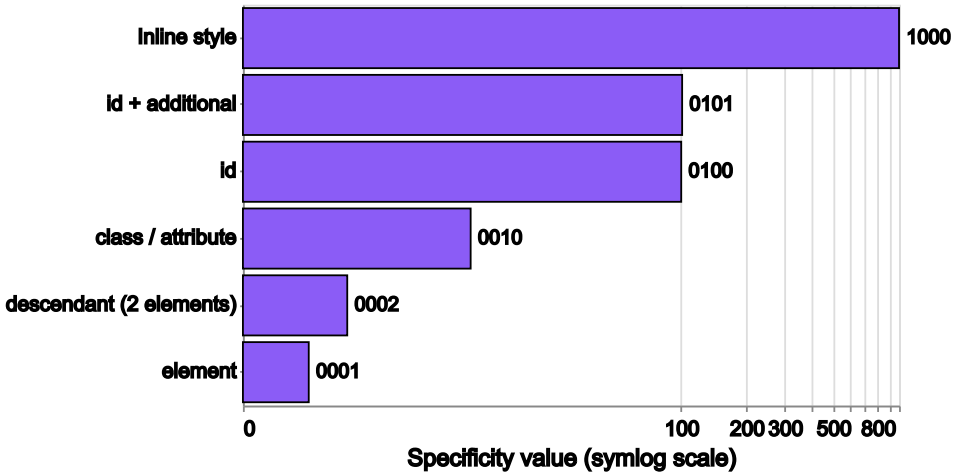
where a = 1 if the rule is an inline style, b = number of id selectors, c = number of class and attribute selectors, and d = number of element selectors.

Check against the table:

$$\text{div \#firstExample}: \; b = 1,\; d = 1 \;\Rightarrow\; S = 100 + 1 = 101$$

$$\text{div form}: \; d = 2 \;\Rightarrow\; S = 2$$

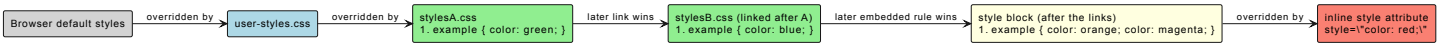
$$\text{.example}: \; c = 1 \;\Rightarrow\; S = 10$$



> **Note for Review** Pro-Tip: The slide's decimal-looking scale is a teaching device. Real browsers compare the columns **left to right** like a tuple, not as one carried number — so even eleven class selectors can never beat one id. Also remember: when specificity ties, **location** (the later rule) decides.

10.4 Location — Who Overrides Whom

When specificity ties or sources differ, **where the style comes from, and in what order it is written**, settles the matter. The figure below runs from weakest to strongest:



The numbered rules in the lecture figure, in plain words:

No.	Rule
1	The user's own `.css` file overrides the browser's default style settings.
2	An **inline** style overrides an external author style sheet or an embedded style sheet.
3	Styles in external author sheet X override those in external sheet Y **if X's `<link>` element comes after Y's** in the HTML document. Whichever is last is the one that sustains.
4	An **embedded** style overrides an **equally specific** rule defined in an external author style sheet **if it appears after** the external sheet's `<link>` element.
5	When the **same property is defined multiple times within a single declaration block, the last one takes precedence.** (Magenta overrides orange.)
6	Finally, the inline `style="color: red;"` on the paragraph overrides all of the above, so the sample text is red.

Putting it all together on the final example:

```
<head>
  <link rel="stylesheet" href="stylesA.css" />
  <link rel="stylesheet" href="stylesB.css" />
  <style>
    #example {
      color: orange;
      color: magenta;
    }
  </style>
</head>
<body>
  <p id="example" style="color: red;">
    sample text
  </p>
</body>
```

Step	Competing declarations	Winner	Reason
1	`stylesA.css` green vs `stylesB.css` blue	blue	same specificity; B is linked later
2	blue (external) vs the `<style>` block	block value	same specificity; embedded appears after the links
3	`orange` vs `magenta` in one block	magenta	last declaration in a block wins
4	magenta vs inline `red`	**red**	inline style (1000) beats the id rule

So whether a rule is internal, external, inline, user-defined or browser-defined, whether it is nearer in proximity, or whether it is simply repeated (the last one persists) — if you know these priorities, you can design a document so that multiple elements take multiple stylings predictably.

> **Note for Review** Pro-Tip: In the standard cascade, ordinary **author** styles beat **user** styles, and both beat browser defaults (the `!important` flag reverses author vs user). The lecture figure only uses user styles to show they beat the browser default, so do not read it as "user always beats author". For exams, the safest summary is: **specificity first, then source/location order (later wins).**

11. Practice Questions

1. **The stubborn paragraph.** A page has this element and stylesheet:

```
<p id="intro" class="lead note">Welcome</p>
```

```
p { color: green; }
.lead { color: blue; }
.note { color: purple; }
div p { color: orange; }
```

   The `p` sits directly inside `body`. (a) Compute the specificity value of each rule and decide the final colour. (b) Swap the order of `.lead` and `.note` — does anything change? (c) Now the `p` is moved into a `div`. Does `div p` win, and why or why not? (d) Which minimal single change would make the text orange *without* touching any existing rule?

2. **Forty pages, one client.** A school website has 40 pages. The client wants every `h1` navy on all pages, except the admission page, whose `h1` must be dark red, and a single `h1` on the home page must be green for one week only. Choose a location (inline, internal, external) for each requirement, justify the scope you need, and explain what ordering of `<link>` and `<style>` you would use so that the exceptions win without raising specificity wildly.

3. **The missing border.** Given

```
div { font-weight: bold; margin: 50px; border: 1pt solid green; }
```

   with two `p` children, predict exactly what each `p` receives. Then add `p { border: inherit; margin: inherit; }`. What changes visually, why does the layout now look "doubled", and why was `font-weight` never part of the problem?

4. **Who gets picked?** For the HTML below, list which of the paragraphs A to E are selected by each selector: `h3+p`, `h3~p`, `div p`, `div>p`, `body>p`. Explain any selector that picks nothing different from another.

```
<body>
  <h3>T</h3>
  <p>A</p>
  <p>B</p>
  <div>
    <p>C</p>
    <section><p>D</p></section>
  </div>
  <h3>U</h3>
  <p>E</p>
</body>
```

5. **Units under pressure.** The root font size is $16\,\text{px}$. A parent box has `font-size: 20px` and `width: 600px`. Inside it, a child has `font-size: 1.5em; padding: 2em; width: 50%;`. (a) Compute the child's font size, padding and width in pixels. (b) Rewrite the font size and padding using `rem` and recompute. (c) Which version stays stable if someone nests the child one level deeper inside another element with `font-size: 30px`, and why? (d) Convert `rgb(255,105,180)` to hexadecimal by hand to show how a designer could write the same colour another way.